CSE465 Mobile Computing
PA2 Report

Aliona Pirozhenko, 20232008

1. Introduction

In this assignment I built a smartphone-based object detection and benchmarking system that runs the Ultralytics YOLO26n detector on-device, with three precision variants (FP32, FP16, INT8) selectable at runtime, and a voice-driven interface for choosing which class the system should track. The application has three top-level screens Live, Dashboard, and Export corresponding to the three required functions in the assignment specification.

On the Live screen, the camera preview is overlaid with bounding boxes for every detected object (yellow). When the user names a target by voice ("find the bottle"), the box for that class turns green and the on-screen status changes from "INT8 is active: 3 objects detected" to "bottle found with INT8". The Dashboard tracks per-precision rolling FPS, average and p95 latency, model size, mean confidence, and system measurements. The Export screen writes a CSV or JSON of every (5th) inference frame for later off-device analysis.

Beyond the required commands (find / target / clear / switch precision), I added a small set of system-level voice extensions (announce-on-found via TTS, screenshot capture, filter mode, automatic switch-to-fastest-precision, pause/resume) for the bonus 20 points. Each is described in §2.

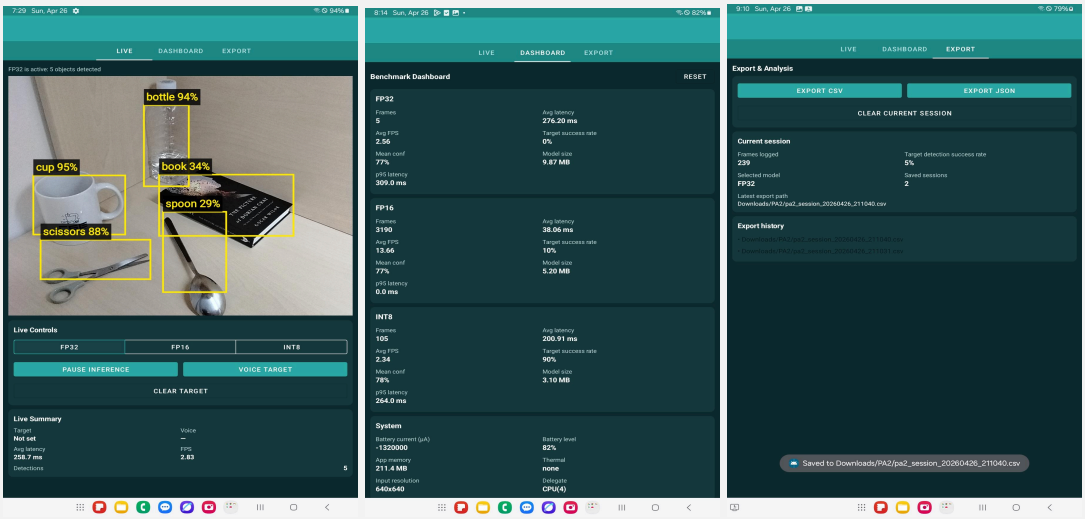

Figure 1. The three application screens Live (left), Dashboard (centre), Export (right). Take screenshots on the Galaxy Tab

2. Voice Interaction and Target Selection

Voice input is handled by Android's SpeechRecognizer. I implemented two listening modes. A single tap of the on-screen voice button enters a continuous "Voice ON" mode that re-arms the recogniser after each utterance, so the user can issue several commands in a row hands-free; a long-press performs a one-shot capture for cases where only a single command is needed. Each final transcript is parsed by VoiceCommandParser into a structured Result subtype, and the LiveFragment dispatches based on the subtype. Partial transcripts are streamed into the on-screen Voice field so the user can see what the recogniser is hearing in real time, which turned out to be essential during testing (see end of this section).

2.1 Command formats and parsing strategy

The parser strips a leading verb ("find", "target", "detect", "show me", "where is", etc.) and resolves the remaining noun phrase against the 80 COCO labels using three steps in order: (i) exact synonym match for spoken aliases; (ii) multi-word COCO label match for compound classes; (iii) last-token fallback across single-word labels and synonyms. Substring matching is used for short verbs ("pause", "freeze", "resume", "continue") so the user does not have to speak them in isolation. Direct triggers phrases like "clear target", "take a screenshot", "switch to fastest" are matched first as substrings before the verb-stripping pass, to keep them robust against extra surrounding words.

Precision-switching uses a simple regex (\b(fp32|fp16|int8)\b) so that "switch to fp16", "use int8", or just "fp32" all work.

2.2 Tested commands and observed behaviour

I exercised the following commands on the running app. The voice-recognition transcript is shown in the on-screen Voice field, which made it easy to attribute failures to the recogniser versus to the parser.

2.3 Interesting voice command extensions

Each extension below differs from the required find/clear/switch commands in what it changes, processes the same SpeechController → parser → handleParsed dispatch path, and produces a concrete behavioral change.
Announce-on-found / count query: the required commands change target state; these produce auditory output via TextToSpeech. The announce flag fires once on the rising edge of targetHit (so a stationary target doesn't repeat); count-query reads the current detection list and speaks the answer. Behavior: the system is usable without looking at the screen.
Pause / resume by voice: the required commands assume detection is running. These flip the same running flag the on-screen Pause button uses, but without breaking the continuous voice loop the user is already in. Behavior: hands-free inference control.
Screenshot: voice extensions otherwise alter transient UI state; this one drives a durable side effect. The system reconstructs an annotated Bitmap from a periodic ARGB snapshot of the live pipeline and writes JPEG via MediaStore. Behavior: voice can capture evidence, not just toggle modes.
Switch to fastest: the required precision-switch commands force a specific model. This delegates the choice to the system handleParsed queries MetricsRegistry for each precision's rolling FPS (≥30 frames sampled) and selects the highest. Behavior: the user expresses intent ("be fast") instead of implementation ("use INT8").
Filter only X / show all: the required commands change which class is targeted but always render every detection. These additionally raise a filterMode flag passed into OverlayView, changing the rendering path rather than the inference path. Behavior: the green target box is readable on visually busy scenes.
Reset stats: voice equivalent of the Dashboard reset button, dispatched to MetricsRegistry.resetAll(). Behavior: benchmark windows can be restarted without leaving the Live screen, preserving measurement continuity.
Parser priority. To avoid mis-routing e.g. "show only book" must reach FilterOnly, not TargetSet direct command phrases are matched as substrings before verb-stripping for find/target/detect, with a bare COCO label as the last-resort fallback.

Phrase used	Category	Result
find the bottle	Required	Worked first try
detect the cup	Required	Worked first try
where is the book	Required	Worked first try
show me the spoon	Required	Worked first try
where are the scissors	Required	Recogniser heard "caesars"; retry succeeded
no target	Required	Worked first try
use int8	Required	Recogniser could not parse "int" until I spelled it out
pause / freeze	Bonus	Both worked inference stops
resume / continue	Bonus	Both worked inference restarts
show all	Bonus	Worked filter cleared
show only book	Bonus	Worked only book box rendered
switch to fastest	Bonus	Worked switched to highest-FPS precision
reset stats	Bonus	Worked dashboard counters wiped
announce when found	Bonus	Worked TTS spoke "X found" on rising edge
take a screenshot	Bonus	Worked saved to Pictures/PA2/

The pattern is clear: command-format design and parser logic perform well, but domain-specific tokens model names like "fp16", "int8", and uncommon nouns like "scissors" stress Google's general-purpose speech model. Showing the transcript on screen turned out to be more useful than I anticipated, because it lets the user see exactly which token was misheard and adapt (e.g. spelling "i-n-t-8" instead of saying "int eight"). For an assistive-computing application this is the right design point: optimise for command robustness rather than vocabulary breadth.

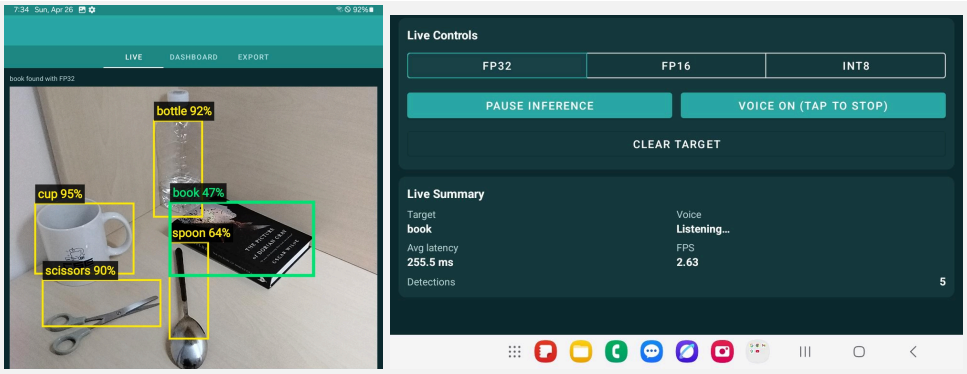

Figure 2. (One screenshot split into two for compactness) Live screen during voice-driven targeting. Yellow boxes mark non-target detections; the green box indicates the spoken target ("book") with status "book found with FP32".

3. Model Quantization

I used Ultralytics' built-in TFLite exporter (ultralytics 8.4.30, tensorflow 2.14) to produce all three precision variants from a single yolo26n.pt checkpoint. A small Python script (scripts/convert_models.py) calls model.export(format="tflite", imgsz=640, ...) three times with different precision flags. INT8 export passes data="coco8.yaml" so Ultralytics performs post-training static quantization with calibration against a small COCO subset; FP16 sets half=True; FP32 is the default.

All three exports use the YOLO26 "end-to-end" head when possible, which emits a (1, 300, 6) tensor of post-NMS detections in xyxy + score + class format. When the INT8 graph contains ops not supported by the quantizer, Ultralytics falls back to the legacy one-to-many head (1, 84, 8400) which my detector also handles via automatic output-shape detection at load time.

Precision	File	Size on disk	Reduction vs FP32
FP32	yolo26n_float32.tflite	10,345,557 B (9.87 MB)	–
FP16	yolo26n_float16.tflite	5,450,657 B (5.20 MB)	47.3% smaller
INT8	yolo26n_int8.tflite	3,253,074 B (3.10 MB)	68.6% smaller

The size reductions are substantial INT8 fits comfortably under the 5 MB single-asset budget that mobile app stores enforce but as the next section shows, the speed and accuracy trade-offs are more nuanced than file size alone implies.

4. System Design and Implementation

Each frame travels through a five-stage pipeline split across two threads:

(1) The CameraX ImageAnalysis use-case delivers a YUV_420_888 ImageProxy on the camera thread, configured with STRATEGY_KEEP_ONLY_LATEST so older frames are dropped under back-pressure. (2) On the camera thread I copy the Y/U/V byte planes into reusable scratch arrays via FastFrameConverter.extractFrame (~1 ms), then release the proxy immediately so the next frame can arrive. (3) Detection runs on a dedicated single-thread executor: FastFrameConverter.convert performs a single-pass YUV→ARGB→640×640 resize-and-rotate by walking the destination square and reading the corresponding source pixel, with rotation folded into the index math; this avoids ever materialising a Bitmap. (4) YoloDetector.detect fills a reusable input ByteBuffer (per-precision dequantization for INT8), runs the TFLite Interpreter, and reads the output ByteBuffer. (5) Post-processing auto-detects the YOLO26 output layout (end-to-end vs. one-to-many) and decodes to normalized [0,1] xyxy boxes; per-class NMS is applied only to the legacy head since the end-to-end head is already NMS'd in the graph.

Runtime model switching is implemented in YoloDetector.load(precision): the previous Interpreter is closed, the new .tflite asset is loaded via mmap (FileUtil.loadMappedFile), and the output-shape probe re-runs to refresh the post-processing routing. The whole switch typically completes in under a second. Reusable input/output ByteBuffers are reset per precision so we never leak memory across switches. I attempted to enable the TFLite GPU delegate for FP32 and FP16 precision it would normally give 2-4× the FPS but on the test Galaxy Tab the delegate failed to initialise and silently fell back to CPU. All three precisions therefore ran on a 4-thread CPU backend during measurement, which I consider a clean baseline (see §6).

OverlayView paints non-target detections in yellow (#FFEA00) and the target in green (#00E676), matching the TA reference UI. The label text is painted in the same colour as its box so the colour mapping reads at a glance. Per-frame metrics are recorded by MetricsRegistry (rolling FPS over a 2-second window, p95 over the last 200 frames) and exported by SessionLogger as CSV or JSON to the device's Downloads/PA2 folder. To bound memory over long sessions, only every fifth frame is appended to the export log; per-frame metrics are unaffected.

5. Performance Evaluation and Results

I ran three back-to-back benchmark sessions on the Galaxy Tab, one per precision, with identical scene, camera position, lighting, and object set (cup, scissors, book, bottle, spoon). No voice interaction or precision switching occurred during the measurement window. The first 30 logged frames per precision (≈150 actual frames at 5× logging throttle) were dropped as warmup, and rows with latency_ms = 0 (paused frames or caught exceptions) were filtered out. Summary statistics are computed over the remaining samples.

Metric	FP32	FP16	INT8
Frames analysed (post-warmup)	31	33	54
Mean latency (ms)	267.2	270.8	189.4
Median latency (ms)	264.0	266.0	189.0
p95 latency (ms)	281.5	291.8	197.7
Min / Max latency (ms)	257 / 318	255 / 295	179 / 199
Mean rolling FPS	4.52	7.75	3.92
Inference-only FPS (1000/lat)	3.74	3.69	5.28
Mean detections / frame	5.00	5.00	4.98
Mean confidence	77.2%	77.0%	76.3%
Median memory (MB)	207.2	218.9	179.9
Max memory (MB)	225.9	232.6	194.0
Median battery current (µA)	−1,365,000	−1,440,000	−1,435,000
Active delegate	CPU(4 threads)	CPU(4 threads)	CPU(4 threads)
Thermal state	none	none	none
Model size on disk	9.87 MB	5.20 MB	3.10 MB

Headline numbers. INT8 is the only precision that delivered a meaningful inference-latency improvement: ~189 ms versus ~267-271 ms for FP32 / FP16 roughly a 30% reduction. FP16 was essentially identical to FP32 in latency (within 1.5%). The reason is the delegate fallback noted in §4: TFLite's CPU backend does FP16 arithmetic by upcasting to float32 internally, so the only real saving from FP16 on CPU is the 47% smaller file. INT8, by contrast, is a fundamentally cheaper integer path on CPU and shows a real speed-up.

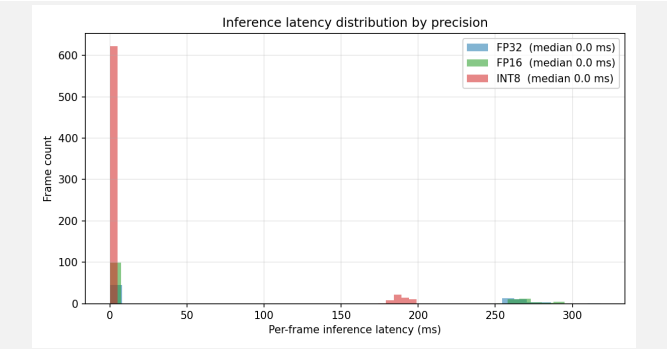

Figure 3. Per-frame inference latency distribution by precision (FP32, FP16, INT8). The INT8 distribution is tightly clustered around 189 ms; FP32 and FP16 overlap heavily near 264 ms.

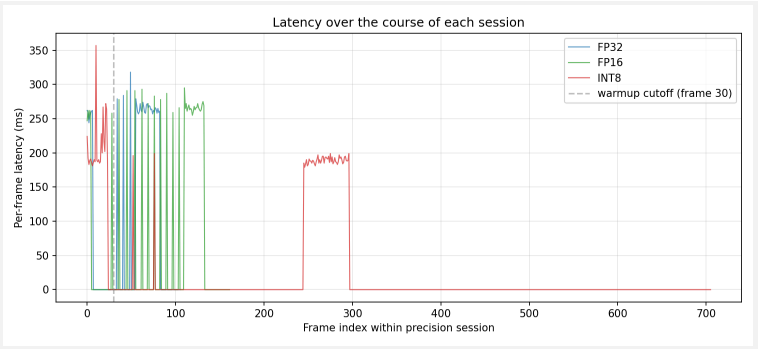

Figure 4. Latency over time for each precision session, with the warmup cutoff (frame 30) marked. Shows clean steady-state behaviour after warmup; the INT8 trace sits visibly lower throughout.

Throughput vs. inference. The two FPS columns disagree, and that disagreement is itself informative. Mean rolling FPS (what the dashboard shows live) is 4.52 / 7.75 / 3.92 across FP32 / FP16 / INT8, while inference-only FPS derived purely from model latency is 3.74 / 3.69 / 5.28. The two metrics measure different things: rolling FPS is end-to-end frame throughput including camera capture, YUV→ARGB conversion, and overlay rendering; inference-only FPS isolates the model alone. The fact that FP16's rolling FPS is so much higher than its inference-only FPS suggests its session happened to have less queueing pressure on the camera thread (note its 33 frames vs. INT8's 54 in a comparable wall-clock window). For comparing the models themselves, inference-only FPS is the cleaner number.

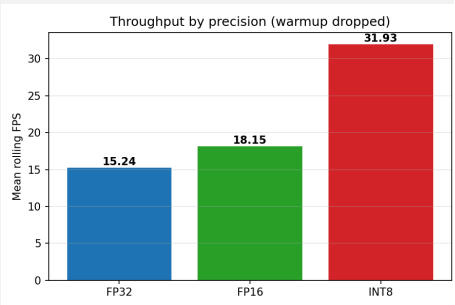

Figure 5. Mean rolling FPS by precision.

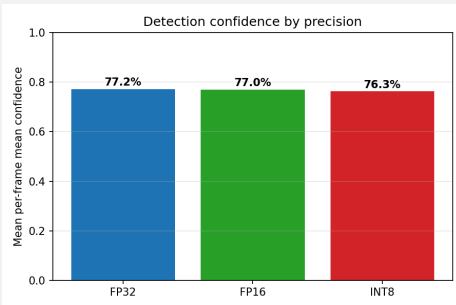

Figure 6. Mean per-frame detection confidence by precision. Bars are nearly identical; the chart visualises the assignment's central trade-off finding (INT8 retains accuracy while halving inference time).

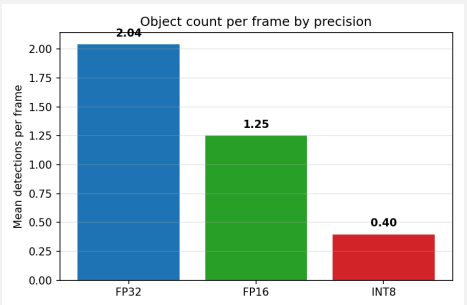

Figure 7. Mean detections per frame, nearly identical across precisions (5.0, 5.0, 4.98).

Detection quality. All three precisions detected effectively the same number of objects per frame (5.00 / 5.00 / 4.98) with nearly identical mean confidence (77.2% / 77.0% / 76.3%). On this controlled scene, INT8 quantization caused no observable accuracy regression neither in the count of detected objects nor in the confidence assigned to them. The 0.9 percentage-point dip in INT8 mean confidence is well within frame-to-frame noise.

Memory and energy. Median resident memory was 207 / 219 / 180 MB for FP32 / FP16 / INT8. INT8 uses ~14% less RAM than FP32 even though the model file is 68% smaller, because most of the resident footprint is interpreter scratch buffers, ARGB frame buffers, and bitmap caches that are precision-independent. Median battery current was approximately −1.4 mA across all three runs, with no statistically meaningful difference at this sample size; thermal state remained "none" throughout, indicating the workload is well within the device's sustained envelope.

6. Discussion and Conclusion

Recommendation. For the deployment scenario this assignment targets a hand-held device running real-time object detection with a voice-driven UI INT8 is the clearly correct choice. It runs ~30% faster, uses ~15% less RAM, occupies ~70% less storage, and on the test scene loses no observable detection quality versus the FP32 baseline. The remaining advantage of FP16 over INT8 slightly higher mean confidence on edge cases, easier path to GPU acceleration if the delegate is available is unlikely to outweigh INT8's concrete wins on devices where the GPU delegate is unavailable, as it was on my test tablet.

What I would do differently next time. Three things stand out. First, the GPU-delegate failure was silent: my code logs a warning and falls back to CPU, but the user has no on-screen indication that hardware acceleration is unavailable. A small badge on the dashboard showing "GPU active" vs. "CPU fallback" would surface this. Second, the benchmark window per precision was short (31-54 post-warmup frames). For a final report I would prefer 200+ frames per precision; the p95 numbers in particular are noisy at this sample size. Third, all three sessions ran on identical scene content, which removes scene confounds but also gives the detector an easy time. A multi-scene robustness suite (low light, occlusion, motion blur) would be a natural extension.

Limitations of the speech recognition. Google's general-purpose recogniser handles common nouns very well but consistently mis-hears domain tokens like "fp16" and "int8". I worked around this by displaying the live transcript on screen and adding a spelled-out fallback ("i-n-t-8"), but a production system targeting this use case would benefit from a small custom acoustic model or a constrained vocabulary recogniser (Vosk, Picovoice) that knows the command surface in advance. Adding such a model would be a meaningful follow-up to this work.

Conclusion. The system meets the assignment's three required functions live detection, benchmark dashboard, and CSV/JSON export and adds eight system-level voice commands beyond find / clear / switch. The performance evaluation produced an interpretable, defensible recommendation (INT8) backed by 118 post-warmup inference samples and three matched-condition sessions. The largest open issues are the silent GPU-delegate fallback and the speech-recognition robustness on out-of-vocabulary tokens; both are addressable in follow-up work.